Comparación de representaciones para detección de cambios

Ejecución de la sección G del protocolo sobre OSCD: Clay v1.5, SatMAE y Prithvi EO 2.0

Gabriel Gómez · Maestría MISIS · Repositorio *geospatial-multimodal-analysis* · Resultados en `results/comparacion_protocolo/`

Resumen

Se extrajeron los patch tokens de la última capa del encoder de los tres modelos, congelados, sobre los 14 pares de OSCD con máscara pública. Para cada token se calculó la distancia coseno entre T1 y T2 y se etiquetó como cambio (C), sin cambio (NC) o mixto según la fracción de píxeles con cambio de su región. Con esas distancias se calcularon el AUROC y el índice de Fisher por par de imágenes (secciones G.1 a G.4).

- **Comparación principal (2 pares comunes: abudhabi, cupertino).** AUROC: Clay 0.902 ± 0.120, SatMAE 0.907 ± 0.124, Prithvi 0.958 ± 0.024. Con dos pares esta comparación no tiene peso estadístico.
- **Clay y SatMAE sobre los 14 pares.** AUROC 0.811 ± 0.170 y 0.843 ± 0.160; Fisher 0.81 ± 1.08 y 1.72 ± 1.81. SatMAE separa algo mejor, con alta variabilidad entre pares en ambos.
- **Prithvi tiene solo 6 tokens C en total.** Sus tokens cubren 480 × 480 m y en OSCD casi ningún token de ese tamaño supera el 50 % de cambio. Solo 2 de sus 5 escenas permiten calcular AUROC, y por eso la comparación principal queda en 2 pares.
- **Clay se evaluó con píxeles sin normalizar** (sección 8). Sus resultados deben considerarse preliminares hasta repetir la extracción con la normalización oficial.

1. Objetivo y alcance

El protocolo pregunta qué tan diferentes son, en el espacio de representación de cada modelo, las zonas que cambiaron frente a las que no, en un escenario zero-shot: los modelos no se entrenan y las etiquetas de OSCD se usan solo para evaluar. Esta ejecución cubre:

- **G.1** Identificación de regiones con cambio, sin cambio y mixtas.
- **G.2** Distancia coseno entre los embeddings de T1 y T2 en cada posición espacial.
- **G.3** AUROC por par de imágenes y modelo (capacidad de discriminación).
- **G.4** Índice de Fisher por par de imágenes y modelo (separabilidad).

La visualización (G.5: histogramas, mapas de distancias y UMAP) queda pendiente. Los datos necesarios ya están guardados, de modo que no hace falta volver a ejecutar los modelos.

2. Configuración de la ejecución

2.1 Modelos y recortes

	Clay v1.5	SatMAE	Prithvi EO 2.0
Checkpoint	clay-v1.5.ckpt	MVRL/satmae-vitlarge-multispec-pretrain (group_c)	prithvi_eo_v2_300 (terratorch)
Bandas usadas	B02–B08, B8A, B11, B12 (10)	B02–B08, B8A, B11, B12 (10), en 3 grupos espectrales	B02, B03, B04, B8A, B11, B12 (6)
Resolución de entrada	10 m	10 m	30 m (promedio de bloques 3×3)
Chip	256 × 256 px	96 × 96 px	224 × 224 px
Huella del chip	2560 m	960 m	6720 m
Parche / grilla de tokens	8 px / 32 × 32	8 px / 12 × 12 (× 3 grupos)	16 px / 14 × 14 (× 2 fechas)

	Clay v1.5	SatMAE	Prithvi EO 2.0
Región por token	80 m	80 m	480 m
Metadatos	GSD, longitudes de onda, lat/lon (por ciudad), semana del año	No	No (versión base)
Normalización	Ninguna (pendiente)	SentinelNormalize oficial (μ ± 2σ → [0, 1])	z-score oficial (config.json)
T1 y T2	Pasadas separadas	Pasadas separadas	Una pasada conjunta (T = 2)
Chips / escenas	75 / 14	634 / 14	5 / 5

2.2 Recorte de la escena

Cada modelo divide la escena con su propia grilla de chips, **sin solape**, centrada en la escena (el borde sobrante se reparte por igual a ambos lados). El número de chips depende del tamaño del chip y de la escena, y por eso varía entre modelos. No se usa padding ni reescalado: una escena más pequeña que la huella del chip no produce chips.

En Prithvi se recorta primero una ventana de 672 × 672 px a 10 m y luego se reduce a 224 × 224 px a 30 m. La máscara recibe la misma reducción. Así la imagen y la máscara cubren exactamente el mismo terreno. Nueve escenas miden menos de 672 px en algún lado y quedan fuera de la evaluación de Prithvi: paris, mumbai, aguasclaras, nantes, rennes, bercy, bordeaux, hongkong, saclay_e.

En los tres modelos se usa la salida de la última capa del encoder y se descarta el token CLS. En SatMAE la distancia de cada token se combina entre los 3 grupos espectrales con el máximo (`combine_mode = max`); también se guarda la distancia de cada grupo.

3. Flujo de procesamiento

La ejecución separa la **extracción**, que es costosa y requiere los modelos, de la **evaluación**, que es rápida y trabaja sobre archivos. Así se pueden recalcular métricas, cambiar umbrales o generar figuras sin volver a pasar las imágenes por los modelos.

3.1 Extracción (`run_all_clay.py`, `run_all_satmae.py`, `run_all_prithvi.py`)

Para cada chip se calculan los embeddings e1 (T1) y e2 (T2), se resumen a nivel de token y se descartan antes de pasar al chip siguiente. Por escena se guarda un archivo `<modelo>/<region>.npz` con:

Campo	Contenido	Uso
dist	Distancia coseno 1 − cos(e1, e2) por token	G.2–G.4, histogramas, mapas
p	Fracción de píxeles con cambio en la región del token	G.1
label	1 = C, 0 = NC, −1 = mixta	G.1–G.4
center_row_10m, center_col_10m	Centro del token en la grilla de 10 m de la escena	Mapas, cruce entre modelos
chip_idx, token_row, token_col	Chip y posición del token dentro del chip	Mapas
delta, delta_token_idx	|e1 − e2| en float16, solo tokens C y NC	UMAP (G.5)
dist_groups (solo SatMAE)	Distancia por grupo espectral	Recombinar grupos

Guardar e1 y e2 completos ocuparía unos 3 GB. Con este esquema la carpeta de resultados ocupa unos 670 MB (Clay 147 MB, SatMAE 521 MB, Prithvi 2 MB). Casi todo corresponde a |e1 − e2| de SatMAE. Los archivos .npz están excluidos de git porque superan el límite de GitHub y se regeneran con los scripts de extracción. Cada modelo escribe además un `manifest.json` con su configuración, los conteos por escena y las escenas excluidas.

3.2 Evaluación (`evaluate.py`)

Lee los .npz de los tres modelos y escribe `evaluacion/por_par.csv` (una fila por modelo y par) y `evaluacion/resumen.json` (conteos, pares válidos e inválidos, y las comparaciones principal y complementaria).

4. Etiquetado de tokens (G.1)

Para cada token, p = píxeles con cambio / píxeles de la región que cubre el token, medido sobre la máscara de OSCD llevada a la grilla nativa del modelo (se remuestrea la máscara, nunca los embeddings). Umbrales del protocolo: **C** si $p > 0.50$, **NC** si $p < 0.05$ y **mixta** (excluida) si $0.05 \leq p \leq 0.50$.

Modelo	Región por token	Tokens	C	NC	Mixtas	% excluido
Clay v1.5	80 m	76 800	1 601	72 497	2 702	3.5 %
SatMAE	80 m	91 296	1 718	86 538	3 040	3.3 %
Prithvi EO 2.0	480 m	980	6	858	116	11.8 %

El desbalance es fuerte en los tres modelos: en Clay y SatMAE alrededor del 2 % de los tokens evaluados son C. En Prithvi la resolución del token domina: con regiones de 480 × 480 m, el cambio urbano de OSCD rara vez ocupa más de la mitad de un token, y aumentan las mixtas (11.8 %).

5. Métricas (G.3 y G.4)

Las métricas se calculan **por par de imágenes**, con todos los tokens C y NC de todos los chips de la escena, y se resumen con la media y la desviación estándar muestral (ddof = 1) entre pares.

- **AUROC**: área bajo la curva ROC, con la distancia coseno como puntaje y C como clase positiva. 0.5 equivale al azar y 1 a una separación perfecta. Un par solo es válido si tiene tokens C y NC.
- **Índice de Fisher**: $F = (\mu_C - \mu_{NC})^2 / (\sigma_C^{\,2} + \sigma_{NC}^{\,2})$, con varianzas poblacionales dentro del par. Si el denominador es cero, el caso se reporta aparte; en esta ejecución no ocurrió.
- **Comparación principal**: solo los pares válidos para los tres modelos, como pide G.3. **Complementaria**: todos los pares válidos de cada modelo.

6. Resultados

6.1 Comparación principal: 2 pares comunes (abudhabi, cupertino)

Modelo	Pares	AUROC (media ± std)	Fisher (media ± std)
Clay v1.5	2	0.902 ± 0.120	0.98 ± 0.80
SatMAE	2	0.907 ± 0.124	3.59 ± 4.04
Prithvi EO 2.0	2	0.958 ± 0.024	1.96 ± 0.04

6.2 Complementaria: todos los pares válidos de cada modelo

Modelo	Pares	AUROC (media ± std)	Fisher (media ± std)
Clay v1.5	14	0.811 ± 0.170	0.81 ± 1.08
SatMAE	14	0.843 ± 0.160	1.72 ± 1.81
Prithvi EO 2.0	2	0.958 ± 0.024	1.96 ± 0.04

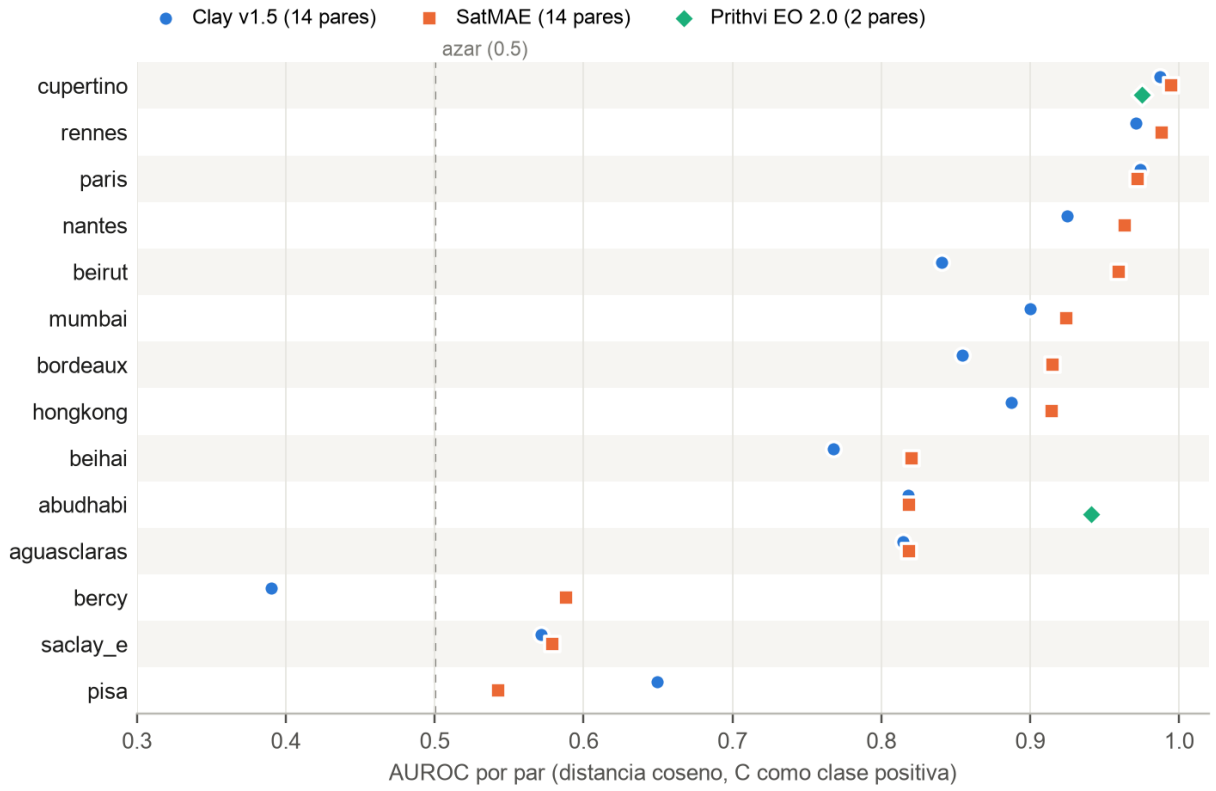


Figura 1. AUROC por par y modelo, con las escenas ordenadas por el AUROC de SatMAE. Prithvi solo aparece en abudhabi y cupertino: en las demás escenas no tiene chips o no tiene tokens C.

6.3 Resultados por par

Par	AUROC Clay	AUROC SatMAE	AUROC Prithvi	Fisher Clay	Fisher SatMAE	Fisher Prithvi	C (Clay / SatMAE / Prithvi)
paris	0.974	0.972	sin chips	4.13	4.03	sin chips	2 / 3 / —
mumbai	0.900	0.924	sin chips	0.85	1.59	sin chips	140 / 138 / —
aguasclaras	0.814	0.818	sin chips	0.21	0.58	sin chips	32 / 39 / —
abudhabi	0.818	0.819	0.941	0.41	0.73	1.99	267 / 267 / 3
nantes	0.925	0.963	sin chips	1.09	2.79	sin chips	28 / 33 / —
rennes	0.971	0.988	sin chips	1.33	2.81	sin chips	28 / 32 / —
bercy	0.390	0.588	sin chips	0.05	0.12	sin chips	9 / 11 / —
bordeaux	0.854	0.915	sin chips	0.17	1.67	sin chips	10 / 16 / —
hongkong	0.887	0.914	sin chips	0.87	1.66	sin chips	162 / 151 / —
pisa	0.649	0.543	sin C	0.08	0.01	sin C	102 / 119 / 0
saclay_e	0.571	0.579	sin chips	0.001	0.02	sin chips	40 / 54 / —
beirut	0.840	0.960	sin C	0.38	0.91	sin C	383 / 416 / 0
beihai	0.768	0.820	sin C	0.22	0.67	sin C	186 / 201 / 0
cupertino	0.987	0.995	0.975	1.55	6.45	1.93	212 / 238 / 3

«sin chips»: la escena es más pequeña que la huella del chip de Prithvi. «sin C»: la escena no tiene ningún token con p > 0.50.

7. Interpretación

- **Clay y SatMAE discriminan el cambio por encima del azar en la mayoría de los pares** (AUROC ≥ 0.6 en 12 de 14 pares con Clay y en 11 de 14 con SatMAE). SatMAE tiene un AUROC medio algo mayor y un índice de Fisher del doble, pero las desviaciones estándar entre pares (≈ 0.16 en AUROC) son mayores

que la diferencia entre ambos, por lo que con 14 pares la diferencia no es concluyente.

- **Las escenas difíciles son las mismas para ambos modelos**: pisa, saclay_e y bercy, con AUROC cercano al azar, y Clay por debajo del azar en bercy (0.39). Esa coincidencia sugiere que la dificultad depende de la escena (tipo de cambio, poca cantidad de tokens C, diferencias radiométricas entre fechas) más que del modelo.
- **El índice de Fisher de SatMAE está dominado por pocas escenas**: va de 0.01 en pisa a 6.45 en cupertino. Por eso su desviación estándar supera a la media, y conviene leerlo junto con el AUROC.
- **El AUROC de Prithvi (0.958) no permite compararlo con los demás**: sale de 6 tokens C repartidos en 2 pares. Es coherente con los otros modelos en esos mismos pares, pero no es una estimación fiable.
- **Las distancias no son comparables en escala entre modelos.** Las distancias medias de Clay son del orden de 0.002–0.04, las de SatMAE de 0.07–0.43 y las de Prithvi de 0.02–0.06. Por eso la comparación se apoya en AUROC y Fisher, que no dependen de la escala absoluta.

8. Limitaciones y diferencias con el protocolo

- **Clay sin normalización de píxeles.** La función `prepare_for_clay` conserva un TODO: no aplica la media y desviación estándar oficiales de `metadata.yaml` (sensor sentinel-2-l2a), y el extractor tampoco lo hace. El punto D pide usar el preprocesamiento recomendado por los autores, así que los resultados de Clay deben repetirse con la normalización oficial antes de usarlos en la selección de modelo.
- **Bandas.** El protocolo (B) indica B02, B03, B04, B08, B11 y B12 para SatMAE y Clay. La ejecución usó 10 bandas en ambos (además B05, B06, B07 y B8A). En SatMAE responde a que el checkpoint group_c se entrenó con esos 3 grupos espectrales; en Clay es una decisión de implementación que el protocolo no contempla.
- **Metadatos de Prithvi.** El protocolo indica que Prithvi usa fechas y coordenadas. El checkpoint base `prithvi_eo_v2_300` no los acepta; la variante que sí lo hace es `prithvi_eo_v2_300_tl`.
- **Parche de Prithvi.** La tabla del punto E indica un parche interno de 14 × 14 px. El parche es de 16 × 16 px; 14 × 14 es la grilla de tokens (224 / 16).
- **Resolución del token.** Los modelos conservan su resolución nativa, como pide G.1, pero eso deja a Prithvi con tokens de 480 m frente a 80 m en los otros dos, y con muy pocos tokens C.
- **Prithvi procesa T1 y T2 juntos.** Sus dos bloques temporales pasan por la misma atención, así que su distancia T1–T2 no es independiente como en Clay y SatMAE, aunque el cálculo sea el mismo.
- **Coordenadas de Clay a nivel de ciudad.** Los archivos de OSCD en torchgeo no traen georreferenciación, así que lat/lon son aproximadas por ciudad y la hora se fija en 10:00.
- **Muestra pequeña.** Son 14 pares en total (5 para Prithvi). Las conclusiones aplican a los pares evaluados y no a la generalización a imágenes nuevas, como indica la sección 7 del protocolo.

9. Decisiones pendientes

- **Normalización de Clay.** Aplicar las estadísticas oficiales y repetir la extracción de Clay. Es prioritario, porque afecta los resultados de uno de los tres modelos.
- **Tratamiento de Prithvi.** Opciones: (a) mantener la configuración y reportar Prithvi como exploratorio, dejando Clay frente a SatMAE sobre 14 pares como comparación sólida; (b) pasarle la imagen a 10 m (tokens de 160 m), que se aparta de su resolución de preentrenamiento; (c) un umbral de C distinto para Prithvi, que se aparta de G.1. La opción (a) es la más fiel al protocolo.
- **Bandas y metadatos.** Ajustar el código al protocolo o el protocolo al código en los puntos de la sección 8. Cualquier cambio de bandas o de checkpoint obliga a repetir la extracción.
- **Visualización (G.5).** Histogramas, mapas de distancias de tres pares y UMAP, a partir de los .npz ya guardados.

10. Reproducibilidad

Desde la raíz del repositorio, con el entorno `.venv` activo. Los scripts de extracción son independientes y pueden ejecutarse en cualquier orden:

```
python run_all_clay.py --oscd-root model_assets/clay/data/oscd \
    --clay-checkpoint model_assets/clay/clay-v1.5.ckpt \
    --clay-metadata model_assets/clay/metadata.yaml
python run_all_satmae.py --oscd-root model_assets/clay/data/oscd
python run_all_prithvi.py --oscd-root model_assets/clay/data/oscd
python evaluate.py
```

Estructura de resultados:

```
results/comparacion_protocolo/
  clay/ satmae/ prithvi/   <region>.npz + manifest.json
  evaluacion/          por_par.csv + resumen.json
```

Archivos de código involucrados: `config.py` (ficha de cada modelo), `preprocessing.py` (bandas, remuestreo, recortes, normalización), `metrics.py` (distancia, p, etiquetas, AUROC, Fisher), `run_pipeline.py` (extracción por chip y guardado por escena), `models/*_extractor.py` (carga y forward de cada modelo) y `evaluate.py`.